PEP 482 – Literature Overview for Type Hints

Author: Łukasz Langa <lukasz at python.org>

Discussions-To: Python-Ideas list (https://mail.python.org/archives/list/python-ideas@python.org/)

Status: Final

Type: Informational

Topic: Typing (../topic/typing/)

Created: 08-Jan-2015

Post-History:

Source: https://github.com/python/peps/blob/main/peps/pep-0482.rst

Last modified: 2025-02-01 08:59:27 UTC

Abstract (#abstract)

This PEP is one of three related to type hinting. This PEP gives a literature overview of related work. The main spec is PEP 484 (../pep-0484/).

Existing Approaches for Python (#existing-approaches-for-python)

mypy (#mypy)

(This section is a stub, since mypy (https://mypy-lang.org) is essentially what we're proposing.)

Reticulated Python (#reticulated-python)

Reticulated Python (https://github.com/mvitousek/reticulated) by Michael Vitousek is an example of a slightly different approach to gradual typing for Python. It is described in an actual academic paper (http://wphomes.soic.indiana.edu/jsiek/files/2014/03/retic-python.pdf) written by Vitousek with Jeremy Siek and Jim Baker (the latter of Jython fame).

PyCharm (#pycharm)

PyCharm by JetBrains has been providing a way to specify and check types for about four years. The type system suggested by PyCharm (https://github.com/JetBrains/python-skeletons#types) grew from simple class types to tuple types, generic types, function types, etc. based on feedback of many users who shared their experience of using type hints in their code.

Others (#others)

TBD: Add sections on pyflakes (https://github.com/pyflakes/pyflakes/), pylint (https://www.pylint.org), numpy (https://www.numpy.org), Argument Clinic (https://docs.python.org/3/howto/clinic.html), pytypedecl (https://github.com/google/pytypedecl), numba (https://numba.pydata.org), obiwan (https://pypi.org/project/obiwan).

Existing Approaches in Other Languages (#existing-approaches-in-other-languages)

ActionScript (#actionscript)

ActionScript (https://livedocs.adobe.com/specs/actionscript/3/) is a class-based, single inheritance, object-oriented superset of ECMAScript. It supports interfaces and strong runtime-checked static typing. Compilation supports a “strict dialect” where type mismatches are reported at compile-time.

Example code with types:

```
package {
  import flash.events.Event;

  public class BounceEvent extends Event {
    public static const BOUNCE:String = "bounce";
    private var _side:String = "none";

    public function get side():String {
      return _side;
    }

    public function BounceEvent(type:String, side:String){
      super(type, true);
      _side = side;
    }

    public override function clone():Event {
      return new BounceEvent(type, _side);
    }
  }
}
```

Dart (#dart)

Dart (https://www.dartlang.org) is a class-based, single inheritance, object-oriented language with C-style syntax. It supports interfaces, abstract classes, reified generics, and optional typing.

Types are inferred when possible. The runtime differentiates between two modes of execution: *checked mode* aimed for development (catching type errors at runtime) and *production mode* recommended for speed execution (ignoring types and asserts).

Example code with types:

```
class Point {
    final num x, y;

    Point(this.x, this.y);

    num distanceTo(Point other) {
        var dx = x - other.x;
        var dy = y - other.y;
        return math.sqrt(dx * dx + dy * dy);
    }
}
```

Hack (#hack)

Hack (https://hacklang.org) is a programming language that interoperates seamlessly with PHP. It provides opt-in static type checking, type aliasing, generics, nullable types, and lambdas.

Example code with types:

```
<?hh
class MyClass {
  private ?string $x = null;

  public function alpha(): int {
    return 1;
  }

  public function beta(): string {
    return 'hi test';
  }
}

function f(MyClass $my_inst): string {
  // Will generate a hh_client error
  return $my_inst->alpha();
}
```

TypeScript (#typescript)

TypeScript (http://www.typescriptlang.org) is a typed superset of JavaScript that adds interfaces, classes, mixins and modules to the language.

Type checks are duck typed. Multiple valid function signatures are specified by supplying overloaded function declarations. Functions and classes can use generics as type parameterization. Interfaces can have optional fields. Interfaces can specify array and dictionary types. Classes can have constructors that implicitly add arguments as fields. Classes can have static fields. Classes can have private fields. Classes can have getters/setters for fields (like property). Types are inferred.

Example code with types:

```
interface Drivable {
    start(): void;
    drive(distance: number): boolean;
    getPosition(): number;
}

class Car implements Drivable {
    private _isRunning: boolean;
    private _distanceFromStart: number;

    constructor() {
        this._isRunning = false;
        this._distanceFromStart = 0;
    }

    public start() {
        this._isRunning = true;
    }

    public drive(distance: number): boolean {
        if (this._isRunning) {
            this._distanceFromStart += distance;
            return true;
        }
        return false;
    }

    public getPosition(): number {
        return this._distanceFromStart;
    }
}
```

Copyright (#copyright)